北京郵電大學

Beijing University of Posts and Telecommunications

BUPT Compilers Project2 2024Fall

Name: 项 枫

Student ID: 2022211570

Class Number: 2022211801

Container Number:

6eca9c2b53e9aaa0c65b6af6613814be05cc0d8ccc63b7df064c4a7230aab09f

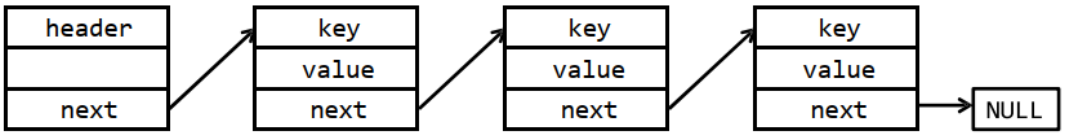
Contents

1 Overview

In the previous project, I have completed the tasks of lexical and syntax analysis, which end up with a syntax tree representation for a syntactically-valid bpl program. Informally, semantic analysis ensures that the program has a well-defined meaning, in other words, via semantic analysis, my compiler finally knows whether an input bpl program is legal or not. Unlike my experience in the previous project, to implement the semantic analyzer, I write all the code by myself. Semantic analysis is not the most difficult task for implementing the bpl compiler, however, it is the most laborious one. To realize the semantic analysis effectively and efficiently, I design various data structures, such as symbol table and data type representation, and carefully consider what is their most appropriate implementation. We have defined bpl tokens by regular expressions and bpl syntax by a context-free grammar. In this project, we will not use more expressive formal languages (i.e., contextsensitive grammar) for semantic analysis. Instead, we use attribute grammar, which assigns each symbol in the CFG a set of actions. Actually, we already used attribute grammar for syntax tree generation during parsing in the previous project. Assigning all symbols with solely synthesized attributes allows me to make a one-pass parser, which builds the syntax tree and performs semantic analysis simultaneously. Running in one-pass is definitely more efficient, though it is preferable to separate these two stages (build the tree and then traverse it to analyze semantics) for better modularity and flexibility. I will start semantic analysis based on the previous project, and will continue the subsequent parts of my bpl compiler on top of this work, hence it is important to keep my code maintainable and extensible.

2 Abstract Data Types

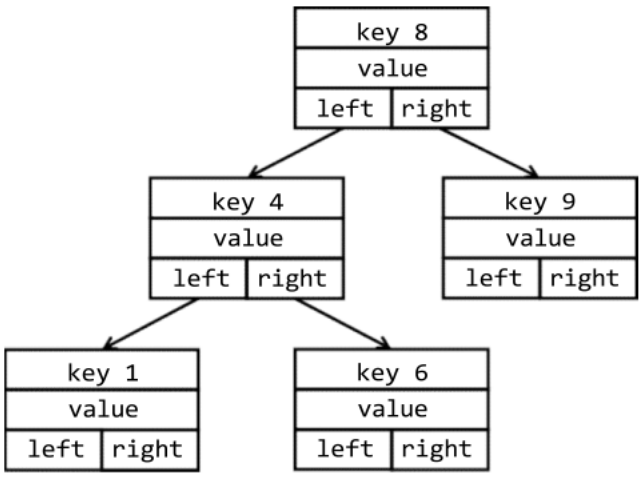
2.1 Linked List



In a linked list symbol table, all symbols are organized in a sequential order. To insert a new symbol, we can simply put it right after the header node, and the time complexity is O(1). However, to lookup a particular symbol, in the worst cases, we need to go through the whole list (O(n) time complexity). Deleting a symbol is the same as lookup, since we should know whether the symbol exists before deleting it.

Clearly, linked list is not efficient for lookup and delete operations. When there are many symbols in the table, these operations can be time-consuming. Nevertheless, the linked list is easy to implement. If you know beforehand that there are only a few symbols during semantic analysis, using a linked list to build the symbol table may be a wise choice.

2.2 Binary Search Tree



Linked list only supports sequential search, while binary tree supports binary search, of which the time complexity to O(log n). A binary search tree node generally has two pointer fields, one to its left child, another to the right one. The structural property of a binary search tree is that, all keys in the left subtree are strictly less than the root's key, while all keys in the right subtree are greater than the root's key. With such a property, a binary search tree is able to lookup/delete in logarithmic time. However, the insert time is also logarithmic, since it should firstly find a suitable position to insert.

However, a binary search tree may degenerate to a linked list, depending on the order of insert and delete. To overcome this limitation, many balance strategies are proposed. Typical variants include AVL tree or red-black tree. These variants propose more strict structural properties to ensure the tree never degenerates, and the operations are in O(log n) time on average. However, implementing a binary search tree with an applicable balance strategy is rather complicated. Binary search tree achieves good trade-off between space and time complexity, hence it is commonly used in some performance-sensitive systems. For example, JDK's TreeMap collection is implemented using red-black tree.

2.3 Hash Table

The time complexity of lookup in a hash table is O(1). A good hash function may provide constant time for all operations (in average). Comparing to other ADTs, hash table is rather simple to implement:

allocate a large consecutive memory space, design a hash function for the given key type (ASCII strings for a symbol table), then insert the key-value at the corresponding position. Note that hash table consumes more space than binary tree, though it is not critical in our project.

A hash function “compresses” the key into a fixed-range index. A good function can produce evenly distributed values with respect to the input string. How to design a good hash function is beyond the scope of our course. You can find many practical examples of hash function in https://github.com/liushoukai/node-hashes. A good candidate is the hash function proposed by P. J. Weinberger.

The hash function inevitably causes collisions, in which multiple keys map to the same index value. To resolve hash collisions, there are generally two approaches: separate chaining and open addressing. The former defines each table element to be the head of a linked list, and appends the collided keys to the same list. The latter is also called rehashing, which re-computes the hash value by alternative hash functions to find next available position. You can also employ other advanced techniques, such as multiplicative hash function or universal hash function, to make your hash table more evenly distributed.

3 Scope Checking

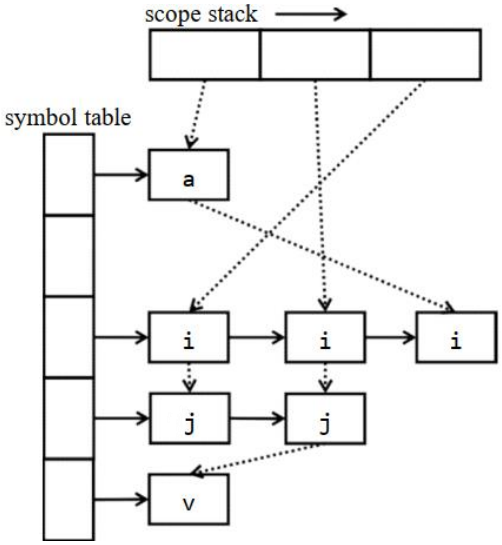
A scope is a section of program text enclosed by basic program delimiters, e . g{}. , in C, or begin-end in Pascal. With scope, variables are only visible within certain sections. Option- ally, our bpl language can provide scoping.

Scope checking refers to the process of determining if an identifier (symbol) is accessible at that location in the program. There are two common approaches to implement a scoping symbol table.

The first approach is imperative-style implementation, in which only one single table is maintained globally. We can implement a hash table with separated chaining, and insert duplicated key to the head of its corresponding linked list. The innermost definition always appears at the head of list. However, when the current scope is enclosed, we should pop out all symbols in the scope. To prevent traversing the whole hash table whenever a scope is closed, you may consider to utilize orthogonal list data structure. In the orthogonal list shown above, three cascading scope is defined. The outermost scope is at the bottom of the stack, in which two symbols, namely a and i are declared. In the second level, symbols i, j and v are declared, and this symbol i shadows the outer declaration. Similar for the top

scope on the stack. When the top scope is popped, we can trace through the list and find that only symbols i and j are defined inside, hence efficiently remove these two keys from the head of their corresponding lists.

Another solution for imperative-style symbol table is assigning each scope with a depth number. Each name in the symbol table is inserted along with the depth number of its containing scope. A name may appear in the symbol table more than once as long as each repetition has a different depth. To lookup a certain name, the symbol table should always return the name with the innermost depth number. Such implementation is similar to an open-addressing hash table.



Another implementation style is so called functional-style, which separates individual symbol table for each scope. Under such implementation, we organize all these symbol tables into a scope stack, the innermost scope is stored at the top of the stack, with the next containing scope that is underneath it, etc.. When a new scope is opened, a new symbol table is created and the variables declared in that scope are inserted into the new table. We then push the symbol table on the scope stack. When a scope is closed, the top symbol table is popped. To find a symbol, we start at the top of the stack and work our way down until we find it. If we do not find it, the variable is not accessible and an error should be generated.

In addition to the obvious overhead of creating/removing symbol tables, the functionalstyle has a disadvantage, that is, all global variables will appear at the bottom of the stack, so scope checking of a program that accesses a lot of global variables can run slowly. Nevertheless, this approach has the advantage that each symbol table, once populated, remains immutable for the rest of the compilation process. Often times, immutable data structures lead to more robust code.

4 Type Checking

4.1 Type System

In programming language, a type is a set of values and a set of operations operating on those values. In bpl, there are two categories of data types: 1) Primitive Types are the base types of the language. These types are provided directly by the underlying hardware. They are int, char and float. 2) Derived Types are constructed by aggregating primitive types or derived types. They are arrays, structures in bpl, and pointers, union in C, etc..

Type checking is the process of verifying that each operation executed in a program respects the type system of the language. This generally means that all operands in any expression are of appropriate types and number. Much of what we do in the semantic analysis phase is type checking. Type checking can be done in compilation, during execution, or divided across both.

A type checker typically should identify the language constructs that have types associated with them, and check them against the predefined semantic rules. In bpl, there are three language constructs: 1) Variable: All variables declared in a bpl program must have a type, either a primitive type or a derived type. 2) Function: Each function has a return type. Each parameter in the function definition has a type, which is also associated with a corresponding argument in a function call. 3) Expression: Each expression has a type based on the type of the variables in the expression, return types of the called functions, or the operands in the expressions.

4.2 Type Equivalence

How to tell whether two types are equivalent? The question is trivial for primitive types: an int is equivalent only to another int, a float only to another float, etc..

However, things get complicated for derived types. Actually, whether two derived types are equivalent depends on how we define “equivalence”. Typically, there are two kinds of type equivalence: named equivalence and structural equivalence. For languages supporting named equivalence, two types are considered equivalent if and only if they have the same name. For example, struct st {int x; } and struct st {char y;}are equivalent types, regardless of their actual definitions. Structural equivalence is more general. Two types are structurally equivalent if a recursive traversal of the two type definition trees leads to completely the same results.

Clearly, checking types for a language supporting named equivalence is easier and quicker than doing that for a language supporting structural equivalence. But the trade-off is, named equivalence does not always reflect what is really being represented in a derived type.

4.3 Derived Types Representation

At implementation level, it is easy to represent primitive types: using constants is sufficient. As for the derived types, arrays and structures, representing them are rather complicated.

For example, we can define arrays of structures, as well as a structure with an array (what's more, a multi-dimensional array!) field.

A common technique is to store the basic information, defining the type as multilevel linked list.

Checking type equivalence is much more complicated for real-world compilers. A rigorous and practical type checker should formally infer types from multilevel language constructs, and support more language features, such as pointers, object-oriented classes, explicit or implicit type coercion, etc.. Though you are not required to implement them, you should be aware of the fact that type checking is generally much more difficult than what you are required to do in this project.

5 Semantic Errors

My analyzer can detect the following semantic errors:

Type 1 variable is used without definition

Type 2 function is invoked without definition

Type 3 variable is redefined in the same scope

Type 4 function is redefined (in the global scope, since we don't have nested function)

Type 5 unmatching types on both sides of assignment operator (=)

Type 6 rvalue on the left side of assignment operator

Type 7 unmatching operands, such as adding an integer to a structure variable

Type 8 the function's return value type mismatches the declared type

Type 9 the function's arguments mismatch the declared parameters (either types or numbers, or both)

Type 10 applying indexing operator ([...]) on non-array type variables

Type 11 applying function invocation operator (foo(...)) on non-function names

Type 12 array indexing with non-integer type expression

Type 13 accessing member of non-structure variable (i.e., misuse the dot operator)

Type 14 accessing an undefined structure member

Type 15 redefine the same structure type

6 Code

The implementation of *head.h* is as follows.

```
#ifndef HEAD_H_  // Prevent multiple inclusions of this header file
#define HEAD_H_

#ifdef __cplusplus  // Allow C++ to call C code by using C linkage
extern "C" {
#endif

#include <stdbool.h>  // Include the standard boolean type for flag values

// Structure for variable declarations
typedef struct {
    char name[100];  // Variable name
    char type[100];  // Variable type
} declare;

// Structure for complex type definitions (arrays, structs, functions)
typedef struct {
    char name[100];          // Type name (e.g., struct, function)
    char elem_type[100];     // Element type for arrays
    declare dec[100];        // Structure members or function local variables
    char return_type[100];   // Function return type
    declare args[100];       // Function arguments
    int arg_num;             // Number of function arguments
    int dec_num;             // Number of declarations in 'dec' array
} definition;

extern declare dec[100];     // Global variable table
extern int dec_num;          // Global variable count
extern definition def[100];  // Complex type definitions array
extern int def_num;          // Complex type definition count
extern char last_type[100];  // Last defined type name

// Function declarations

// Set the name of the last defined type
void set_last_type(char *type);
```

```
// Add a new variable declaration
// Returns true if successful
bool add_dec(char *name, char *type, int lineno);

// Add a new array definition
void add_array(char *name, char *elem_type, int lineno);

// Add a new function or struct definition
// Returns true if successful
bool add_func_or_struct(char *name, int lineno, char *type);

// Link a function or struct to its variables/members
void build_func_and_struct_dec(char *name, char *dec_name, char *type);

// Find the type of a variable, function, or struct
char* find_id_type(char *name);

// Find the declaration of a function or struct by name
declare* find_func_or_struct_dec(char *name);

// Find the return type of a function by name
char* find_func_return_type(char *name);

// Add a new member to a struct
void add_struct_member(char *struct_name, char *member_name, char *member_type, int
lineno);

// Add a new argument to a function
void add_func_args(char *func_name, char *arg_name, char *arg_type, int lineno);

// Add a new variable to a function
void add_func_variables(char *func_name, char *variable_name, char *variable_type, int
lineno);

// Find the type of a struct member
char* find_structure_member_type(char *struct_name, char *member_name);

// Set the return type of a function
void set_func_return_type(char *func_name, char *return_type);

#ifdef __cplusplus
}
#endif
```

```
#endif /* MYHEAD_H_ */
```

The implementation of *head.cpp* is as follows.

```
#include "head.h"
#include <cstdio>
#include <cstring>

declare dec[100];  // Global variable table
int dec_num = 0;   // Number of global variables
definition def[100];  // Array of complex type definitions (array, struct, function)
int def_num = 0;      // Number of complex type definitions (array, struct, function)
char last_type[100];  // Name of the last defined type

// Set the name of the last defined type
void set_last_type(char *type) {
    strcpy(last_type, type);  // Copy the type name to last_type
}

// Add a new variable declaration to the global variable table
// Returns true if successful, false if the variable is already defined
bool add_dec(char *name, char *type, int lineno) {
    // Check if the variable has already been defined
    for (int i = 0; i < dec_num; ++i) {
        if (strcmp(dec[i].name, name) == 0) {
            // Determine the type of the existing definition (function, structure, or
variable)
            const char *errorType =
                strcmp(dec[i].type, "function") == 0 ? "function" :
                strcmp(dec[i].type, "structure") == 0 ? "structure" :
                "variable";
            int errorTypeCode = strcmp(errorType, "function") == 0 ? 4 :
                                strcmp(errorType, "structure") == 0 ? 15 :
                                3;
            // Print error message if variable is redefined
            printf("Error type %d at Line %d: Redefined %s \"%s\".\n", errorTypeCode,
lineno, errorType, name);
            return false;
        }
    }

    // Add new variable declaration
    strcpy(dec[dec_num].name, name);
    strcpy(dec[dec_num].type, type);
```

```
    dec_num++;  // Increase the variable count
    return true;
}

// Add a new array definition to the complex type definitions
void add_array(char *name, char *elem_type, int lineno) {
    // Check if the array has already been defined
    for (int i = 0; i < def_num; ++i) {
        if (strcmp(def[i].name, name) == 0) {
            // Print error if the array is already defined
            printf("Error type 3 at Line %d: Redefined variable \"%s\".\n", lineno,
name);
            return;
        }
    }

    // Add new array definition
    strcpy(def[def_num].name, name);
    strcpy(def[def_num].elem_type, elem_type);
    def[def_num].dec_num = 0;  // Initialize the count of internal variables in the array
    def_num++;  // Increase the type definition count
}

// Add a new function or struct definition to the complex type definitions
// Returns true if successful, false if already defined
bool add_func_or_struct(char *name, int lineno, char *type) {
    // Check if the function or struct has already been defined
    for (int i = 0; i < def_num; ++i) {
        if (strcmp(def[i].name, name) == 0) {
            // Print error based on whether it's a function or a structure
            int errorTypeCode = strcmp(type, "function") == 0 ? 4 : 15;
            const char *typeText = strcmp(type, "function") == 0 ? "function" :
"structure";
            printf("Error type %d at Line %d: Redefined %s \"%s\".\n", errorTypeCode,
lineno, typeText, name);
            return false;
        }
    }

    // Add new function or struct definition
    strcpy(def[def_num].name, name);
    if (strcmp(type, "function") == 0) {
        def[def_num].arg_num = 0; // Initialize function argument count
    }
```

```
    def[def_num].dec_num = 0; // Initialize member or local variable count
    def_num++;  // Increase the type definition count
    return true;
}

// Link function or struct to its internal variables or members
void build_func_and_struct_dec(char *name, char *dec_name, char *type) {
    // Find the specific function or struct by name
    for (int i = 0; i < def_num; ++i) {
        if (strcmp(def[i].name, name) == 0) {
            // Add the member or variable to the function/struct
            strcpy(def[i].dec[def[i].dec_num].name, dec_name);
            strcpy(def[i].dec[def[i].dec_num].type, type);
            def[i].dec_num++;  // Update member count
            return;
        }
    }
}

// Find the type of an identifier (variable, function, or struct)
char* find_id_type(char *name) {
    // Traverse the variable table and find the matching type
    for (int i = 0; i < dec_num; ++i) {
        if (strcmp(dec[i].name, name) == 0) {
            return dec[i].type; // Return the type of the found variable
        }
    }
    return NULL; // Return NULL if not found
}

// Find the declaration of a function or struct by its name
declare* find_func_or_struct_dec(char *name) {
    // Traverse the definition array and find the matching function or struct
    for (int i = 0; i < def_num; i++) {
        if (strcmp(def[i].name, name) == 0) {
            return def[i].dec; // Return the declarations of the found function or struct
        }
    }
    return NULL; // Return NULL if not found
}

// Find the return type of a function by its name
char* find_func_return_type(char *name) {
    // Traverse the definition array and find the matching function
```

```
    for (int i = 0; i < def_num; i++) {
        if (strcmp(def[i].name, name) == 0) {
            return def[i].return_type; // Return the function's return type
        }
    }
    return NULL; // Return NULL if not found
}

// Add a new member to a struct definition
void add_struct_member(char *struct_name, char *member_name, char *member_type, int
lineno) {
    // Traverse the definition array to find the specified struct
    for (int i = 0; i < def_num; i++) {
        if (strcmp(def[i].name, struct_name) == 0) {
            // Check if the member is already defined in the struct
            for (int j = 0; j < def[i].dec_num; j++) {
                if (strcmp(def[i].dec[j].name, member_name) == 0) {
                    // Print error if member is redefined
                    printf("Error type 15 at Line %d: Redefined field \"%s\".\n", lineno,
member_name);
                    return;
                }
            }
            // Add the new member to the struct definition
            strcpy(def[i].dec[def[i].dec_num].name, member_name);
            strcpy(def[i].dec[def[i].dec_num].type, member_type);
            def[i].dec_num++; // Update member count
            return; // Exit after successful addition
        }
    }
}

// Add a new argument to a function
void add_func_args(char *func_name, char *arg_name, char *arg_type, int lineno) {
    // Traverse the definition array to find the specified function
    for (int i = 0; i < def_num; i++) {
        if (strcmp(def[i].name, func_name) == 0) {
            // Check if the argument is already defined in the function
            for (int j = 0; j < def[i].arg_num; j++) {
                if (strcmp(def[i].args[j].name, arg_name) == 0) {
                    // Print error if argument is redefined
                    printf("Error type 3 at Line %d: Redefined variable \"%s\".\n",
lineno, arg_name);
                    return;
```

```
                }
            }
            // Add the new argument to the function definition
            strcpy(def[i].args[def[i].arg_num].name, arg_name);
            strcpy(def[i].args[def[i].arg_num].type, arg_type);
            def[i].arg_num++; // Update argument count
            return; // Exit after successful addition
        }
    }
}

// Add a variable to the list of a function's local variables
void add_func_variables(char *func_name, char *variable_name, char *variable_type, int
lineno) {
    // Traverse the list of defined functions to find the specified function
    for (int i = 0; i < def_num; i++) {
        if (strcmp(def[i].name, func_name) == 0) {
            // Traverse the function's local variables to check if the variable already
exists
            for (int j = 0; j < def[i].dec_num; j++) {
                if (strcmp(def[i].dec[j].name, variable_name) == 0) {
                    // If the variable already exists, print an error and return
                    printf("Error type 3 at Line %d: Redefined variable \"%s\".\n",
lineno, variable_name);
                    return; // Exit after detecting the redefinition
                }
            }
            // If the variable is not defined yet, add it to the function's list of local
variables
            strcpy(def[i].dec[def[i].dec_num].name, variable_name);
            strcpy(def[i].dec[def[i].dec_num].type, variable_type);
            def[i].dec_num++; // Increment the count of local variables
            return; // Exit after successfully adding the variable
        }
    }
}

// Find the type of a member in a struct by its name
char* find_structure_member_type(char *struct_name, char *member_name) {
    // Traverse the list of definitions to find the specified struct
    for (int i = 0; i < def_num; i++) {
        if (strcmp(def[i].name, struct_name) == 0) {
            // Traverse the struct's members to find the specified member
            for (int j = 0; j < def[i].dec_num; j++) {
```

```
                if (strcmp(def[i].dec[j].name, member_name) == 0) {
                    return def[i].dec[j].type; // Return the type of the found member
                }
            }
            return NULL; // Return NULL if the specified member is not found
        }
    }
    return NULL; // Return NULL if the specified struct is not found
}

// Set the return type of a function by its name
void set_func_return_type(char *name, char *type) {
    // Traverse the list of defined functions to find the specified function
    for (int i = 0; i < def_num; i++) {
        if (strcmp(def[i].name, name) == 0) {
            // Set the return type for the found function
            strcpy(def[i].return_type, type);
            return; // Exit after successfully setting the return type
        }
    }
}
```

The implementation of *lex.l* is as follows.

```
%{
    #include "syntax.tab.h"     // Include the parser header
    #include <stdlib.h>          // Standard library for memory allocation
    #include <stdio.h>           // Standard I/O for printing errors
    #include <string.h>          // String handling functions

    #define MAX_ID_SIZE 256     // Maximum size for identifiers

    int line = 1;               // Current line number for error reporting

    // Define the Node structure representing a node in the syntax tree
    typedef struct Head
    {
        int terminal;            // Indicates whether the node is a terminal (leaf) or
non-terminal
        char type[200];          // Type of the node (e.g., "int", "ID", etc.)
        char id[200];            // Identifier for the node (e.g., variable name or type)
        char value_type[200];    // Used for marking the expression type (for EXP)
        int rvalue;              // Used to mark if the node is a right-hand value
        int lineno;              // Line number where the node appears in the source code
        struct Head* child;      // Pointer to the child node (if any)
```

```
        struct Head* next;       // Pointer to the next sibling node (if any)
    } Node;

    // Function to create a new Node in the syntax tree
    Node* create_Node(int terminal, char* type, char* id, Node* child, Node* next, int
lineno)
    {
        Node* newNode = malloc(sizeof(Node));  // Allocate memory for a new node
        if (newNode == NULL) {
            perror("Memory allocation error");  // Error handling if memory allocation
fails
            exit(EXIT_FAILURE);
        }

        newNode->terminal = terminal;  // Set the terminal value
        newNode->lineno = (lineno == 0 && child != NULL) ? child->lineno : lineno;  //
Set line number, prefer child line number if it's zero

        // Copy the type string into the type field, ensuring no overflow
        if (type != NULL) {
            strncpy(newNode->type, type, sizeof(newNode->type) - 1);
            newNode->type[sizeof(newNode->type) - 1] = '\0';  // Null-terminate the
string
        } else {
            newNode->type[0] = '\0';  // Set type to empty if not provided
        }

        // Copy the id string into the id field, ensuring no overflow
        if (id != NULL) {
            strncpy(newNode->id, id, sizeof(newNode->id) - 1);
            newNode->id[sizeof(newNode->id) - 1] = '\0';  // Null-terminate the string
        } else {
            newNode->id[0] = '\0';  // Set id to empty if not provided
        }

        newNode->child = child;  // Set the child pointer
        newNode->next = next;    // Set the next pointer
        newNode->rvalue = 0;     // Initialize rvalue as 0 (not a right-hand value)

        return newNode;  // Return the newly created node
    }

    // Function to print the syntax tree
    void print_tree(Node* root, int depth)
```

```
    {
        if (root == NULL) {
            return;  // Return if the node is NULL
        }

        // Indentation for non-empty nodes
        if (strcmp(root->type, "empty") != 0) {
            for (int i = 0; i < depth; i++) {
                printf("  ");  // Print spaces for indentation
            }
        }

        // Print information about terminal (leaf) nodes
        if (root->terminal == 1) {
            printf("%s", root->type);

            // For specific types of nodes, print additional information (like ID or
value)
            if (strcmp(root->type, "ID") == 0 || strcmp(root->type,"TYPE") == 0 ||
                strcmp(root->type, "INT") == 0 || strcmp(root->type,"FLOAT") == 0 ||
                strcmp(root->type, "CHAR") == 0) {
                printf(": %s", root->id);  // Print the identifier or value of the
terminal node
            }
            printf("\n");
        } else if (strcmp(root->type, "empty") != 0) {
            // For non-terminal, non-empty nodes, print the type and line number
            printf("%s (%d)\n", root->type, root->lineno);
        }

        // Recursively print the child node with increased indentation
        print_tree(root->child, depth + 1);

        // Recursively print the sibling nodes without changing indentation
        print_tree(root->next, depth);
    }
%}

%option noyywrap

int_        [0-9]+|0x[0-9a-fA-F]+
float_      [0-9]+"."[0-9]+
id_         [a-zA-Z_][a-zA-Z0-9_]*
char_       \'.\'|\'\\x[0-9a-fA-F]{2}\'
```

```
%%
"int"                { yylval.node=create_Node(1,"TYPE",yytext,NULL,NULL,line);return
TYPE; }
"float"              { yylval.node=create_Node(1,"TYPE",yytext,NULL,NULL,line);return
TYPE; }
"char"               { yylval.node=create_Node(1,"TYPE",yytext,NULL,NULL,line);return
TYPE; }
"struct"             { yylval.node=create_Node(1,"STRUCT",yytext,NULL,NULL,line);return
STRUCT; }
"if"                 { yylval.node=create_Node(1,"IF",yytext,NULL,NULL,line);return
IF; }
"else"               { yylval.node=create_Node(1,"ELSE",yytext,NULL,NULL,line);return
ELSE; }
"while"              { yylval.node=create_Node(1,"WHILE",yytext,NULL,NULL,line);return
WHILE; }
"return"             { yylval.node=create_Node(1,"RETURN",yytext,NULL,NULL,line);return
RETURN; }
{int_}              {yylval.node=create_Node(1,"INT",yytext,NULL,NULL,line);return INT;}
{float_}             { yylval.node=create_Node(1,"FLOAT",yytext,NULL,NULL,line);return
FLOAT; }
{id_}               { yylval.node=create_Node(1,"ID",yytext,NULL,NULL,line);return ID; }
{char_}             { yylval.node=create_Node(1,"CHAR",yytext,NULL,NULL,line);return
CHAR; }

"."                 { yylval.node=create_Node(1,"DOT",yytext,NULL,NULL,line);return
DOT; }
";"                 { yylval.node=create_Node(1,"SEMI",yytext,NULL,NULL,line);return
SEMI; }
","                 { yylval.node=create_Node(1,"COMMA",yytext,NULL,NULL,line);return
COMMA; }
"="                 { yylval.node=create_Node(1,"ASSIGN",yytext,NULL,NULL,line);return
ASSIGN; }
"<"                 { yylval.node=create_Node(1,"LT",yytext,NULL,NULL,line);return LT; }
">"                 { yylval.node=create_Node(1,"GT",yytext,NULL,NULL,line);return GT; }
"<="                { yylval.node=create_Node(1,"LE",yytext,NULL,NULL,line);return LE; }
">="                { yylval.node=create_Node(1,"GE",yytext,NULL,NULL,line);return GE; }
"!="                { yylval.node=create_Node(1,"NE",yytext,NULL,NULL,line);return NE; }
"=="                { yylval.node=create_Node(1,"EQ",yytext,NULL,NULL,line);return EQ; }
"+"                 { yylval.node=create_Node(1,"PLUS",yytext,NULL,NULL,line);return
PLUS; }
"-"                 { yylval.node=create_Node(1,"MINUS",yytext,NULL,NULL,line);return
MINUS; }
```

```
"*"                { yylval.node=create_Node(1,"MUL",yytext,NULL,NULL,line);return
MUL; }
"/"                { yylval.node=create_Node(1,"DIV",yytext,NULL,NULL,line);return
DIV; }
"&&"               { yylval.node=create_Node(1,"AND",yytext,NULL,NULL,line);return
AND; }
"||"               { yylval.node=create_Node(1,"OR",yytext,NULL,NULL,line);return OR; }
"!"                { yylval.node=create_Node(1,"NOT",yytext,NULL,NULL,line);return
NOT; }
"("                { yylval.node=create_Node(1,"LP",yytext,NULL,NULL,line);return LP; }
")"                { yylval.node=create_Node(1,"RP",yytext,NULL,NULL,line);return RP; }
"["                { yylval.node=create_Node(1,"LB",yytext,NULL,NULL,line);return LB; }
"]"                { yylval.node=create_Node(1,"RB",yytext,NULL,NULL,line);return RB; }
"{"                { yylval.node=create_Node(1,"LC",yytext,NULL,NULL,line);return LC; }
"}"                { yylval.node=create_Node(1,"RC",yytext,NULL,NULL,line);return RC; }

"/*"([^*]|[*][^/]|[*][*]+[^*/])*"*/"  {/* ignore comments */}
[ \t\r]+           {/* ignore whitespace */}
"\n"    {line++;}

0x([0-9a-fA-F]|[g-zG-Z])+     { yylval.node=create_Node(1,"ILLEGAL_HEX_INT", yytext,
NULL, NULL, line); return ILLEGAL_HEX_INT; }
[0-9]{id_}         { yylval.node=create_Node(1,"ILLEGAL_ID", yytext, NULL, NULL, line);
return ILLEGAL_ID; }
'\\x.*'                        { yylval.node=create_Node(1,"ILLEGAL_CHAR", yytext, NULL,
NULL, line); return ILLEGAL_CHAR; }
.                  { yylval.node = create_Node(1,"ILLEGAL", yytext, NULL, NULL, line);
return ILLEGAL; }
%%
```

The implementation of syntax.y is as follows.

```
%{
    #include "lex.yy.c"

    #include <stdio.h>
    #include <stdlib.h>
    #include <string.h>
    #include <stdarg.h>

    #include "head.h"

    void yyerror();

    Node* root=NULL;
    int flag=0;
```

```
// Connect nodes in a sequence
void connect_to_next(int next_num, ...)
{
    if (next_num <= 0) {
        // If there are no nodes or the number of arguments is negative, do nothing
        return;
    }

    va_list valist;
    va_start(valist, next_num);

    Node* temp = va_arg(valist, Node*); // Get the first node
    Node* prev = temp;
    for (int i = 1; i < next_num; i++)
    {
        temp = va_arg(valist, Node*); // Get the next node
        if (prev) {
            prev->next = temp;
        }
        prev = temp; // Update
    }

    if (temp) {
        temp->next = NULL;
    }

    va_end(valist);
}

// Flag to check if currently inside a function body
bool is_in_function=false;
// Store the name of the current function
char *current_function_name;
// Flag to check if there is an array index error
bool index_error=false;
void find_compst_return(Node *Compst, char *expected_type);
void check_nested_statements(Node *stmt, char *expected_type);

/* Struct operations */
// Insert a variable declaration into the symbol table
void insert_vardec(Node *node) {
    if (!node->child->next) {
        // If the child node does not have a next member, it is not an array type
```

```
            add_dec(node->child->id, last_type, node->lineno);
        } else {
            // If the child node has a next member, it is an array type
            if (add_dec(node->child->child->id, "array", node->lineno)) {
                add_array(node->child->child->id, last_type, node->lineno);
            }
        }
    }

    // Insert an extended declaration list into the symbol table
    void insert_ExtDecList(Node* node) {
        // Process the current variable declaration
        insert_vardec(node->child);
        if (node->child->next) {
            // If there are multiple variable declarations (separated by commas),
recursively process the remaining declarations
            insert_ExtDecList(node->child->next->next);
        }
    }

    // Add struct member definitions to the symbol table
    void add_struct_member_deflist(Node* node, char* struct_name)
    {
        // Process member declarations inside a struct
        if(!node||!node->child)
        {
            return;
        }
        Node* temp=node->child;
        Node* temp2=temp->child;
        char *type;
        if (strcmp(temp2->child->type,"TYPE")==0)
        {
            // Not a struct
            type=temp2->child->id;
        }
        else
        {
            // Is a struct
            type=temp2->child->child->next->id;
        }
        Node *temp3=temp->child->next;
        while(temp3)
        {
```

```
            Node* temp4=temp3->child;
            Node* temp5=temp4->child;
            if(temp5->child->next==NULL)
            {
                // Not an array
                add_struct_member(struct_name,temp5->child->id,type,temp5->lineno);
            }
            else
            {
                // Is an array
                add_struct_member(struct_name,temp5->child->child->id,"array",temp5->line
no);
                add_array(temp5->child->child->id,type,temp5->lineno);
            }
            if(temp3->child->next)
            {
                temp3=temp3->child->next->next;
            }
            else
            {
                break;
            }
        }
        if(node->child)
        {
            add_struct_member_deflist(node->child->next,struct_name);
        }
        return;
    }

    /* Function argument and variable insertion */
    // Insert function arguments into the symbol table
    void insert_function_args(char *func_name, Node *node) {
        if (node == NULL) {
            // If the node is empty, there are no more variables to process, return
            return;
        }

        // Process the current ParamDec node
        Node *param = node->child;
        Node *specifier = param->child;
        Node *vardec = specifier->next;
        char *type = specifier->child->id; // Get the type
        char *name = vardec->child->id;    // Get the variable name
```

```
        // Assume arguments are not array types for simplicity
        add_func_args(func_name, name, type, param->lineno);

        // If there is a COMMA, it means there are more arguments, recursively process
the subsequent VarList
        Node *nextParam = node->child->next;
        if (nextParam != NULL && strcmp(nextParam->type, "COMMA") == 0) {
            insert_function_args(func_name, nextParam->next);
        }
    }

    // Insert function variables into the symbol table
    void insert_func_variables(Node *Dec) {
        // If not inside a function or input node is empty, no need to process
        if (!is_in_function || Dec == NULL) {
            return;
        }

        // Inside the function body
        Node *VarDec = Dec->child;
        char *var_name;
        char *var_type;

        // Check if it's an array type
        if (VarDec->child->next) {
            // Handle array type variables
            var_name = VarDec->child->child->id;
            var_type = "array";
            add_func_variables(current_function_name, var_name, var_type,
VarDec->lineno);
            add_array(var_name, last_type, VarDec->lineno);
        } else {
            // Handle regular variables
            var_name = VarDec->child->id;
            var_type = last_type;
            add_func_variables(current_function_name, var_name, var_type,
VarDec->lineno);
        }

        // Check assignment statements and verify type matching
        Node *Assign = VarDec->next;
        if (Assign && strcmp(Assign->type, "ASSIGN") == 0) {
            Node *Exp = Assign->next;
```

```
            if (strcmp(Exp->value_type, "error") != 0 && strcmp(Exp->value_type,
last_type) != 0) {
                printf("Error type 5 at line %d: Type mismatched for assignment\n",
Exp->lineno);
            }
        }
    }

    /* Expression checking */
    // Check if an expression matches the expected type
    void check_type(Node *exp, const char *expected_type, int error_type, int lineno) {
        if (strcmp(exp->value_type, "error") == 0) {
            // If the type is already an error, don't repeat the error message
            return;
        }
        if (expected_type != NULL && strcmp(exp->value_type, expected_type) != 0) {
            printf("Error type %d at line %d: Type mismatched for operands\n",
error_type, lineno);
        }
    }

    // Check if a single expression is of type int or float
    void check_single_arithmetic_exp(Node *exp) {
        check_type(exp, NULL, 7, exp->lineno);
        if (strcmp(exp->value_type, "int") != 0 && strcmp(exp->value_type, "float") != 0)
{
            printf("Error type 7 at line %d: Type mismatched for operands, expected int
or float\n", exp->lineno);
        }
    }

    // Check if a single expression is of type int
    void check_single_logical_exp(Node *exp) {
        check_type(exp, "int", 7, exp->lineno);
    }

    // Check if two expressions in an arithmetic operation are of compatible types
    void check_exp_arithmetic_symbols(Node *exp1, Node *exp2) {
        check_type(exp1, NULL, 7, exp1->lineno);
        check_type(exp2, NULL, 7, exp2->lineno);
        if ((strcmp(exp1->value_type, "int") != 0 && strcmp(exp1->value_type, "float") !=
0) ||
            (strcmp(exp2->value_type, "int") != 0 && strcmp(exp2->value_type, "float") !=
0)) {
```

```
            printf("Error type 7 at line %d: Type mismatched for operands, expected int
or float\n", exp1->lineno);
        }
        if (strcmp(exp1->value_type, exp2->value_type) != 0) {
            printf("Error type 7 at line %d: Type mismatched for operands, expected same
type\n", exp1->lineno);
        }
    }

        // Check if two expressions in a logical operation are of type int
    void check_exp_logical_symbols(Node *exp1, Node *exp2) {
        // Assume logical operations can only be performed by int type variables
        check_type(exp1, "int", 7, exp1->lineno);
        check_type(exp2, "int", 7, exp2->lineno);
    }

    // Check if the left and right sides of an assignment match in type
    void check_exp_assign(Node *exp1, Node *exp2) {
        // Check the type of the left side of the assignment
        check_type(exp1, NULL, 6, exp1->lineno);
        check_type(exp2, NULL, 5, exp2->lineno);
        if (exp1->rvalue) {
            printf("Error type 6 at line %d: rvalue on the left side of assignment
operator\n", exp1->lineno);
            return;
        }
        if (strcmp(exp1->value_type, exp2->value_type) != 0) {
            printf("Error type 5 at line %d: Type mismatched for assignment\n",
exp1->lineno);
        }
    }

    // Check if an expression matches a specific expected type
    void check_Exp_with_specific_type(Node *Exp, char *expected_type) {
        // Check if the type of Exp is as expected
        check_type(Exp, expected_type, 7, Exp->lineno);
    }

    // Check the definitions for type matching in assignments
    void check_Def(Node *Def) {
        // The expected_type is the type defined
        char *expected_type = last_type;
        Node *DecList = Def->child->next;
```

```
        // Process all declarations in the declaration list
        while (DecList) {
            Node *Dec = DecList->child;
            Node *VarDec = Dec->child;

            // If there is an assignment (ASSIGN), check if the expression (Exp) type
matches
            Node *Assign = VarDec->next;
            if (Assign && strcmp(Assign->type, "ASSIGN") == 0) {
                Node *Exp = Assign->next;
                if (strcmp(Exp->value_type, "error") != 0 && strcmp(Exp->value_type,
expected_type) != 0) {
                    printf("Error type 5 at line %d: Type mismatched for assignment\n",
Exp->lineno);
                }
            }

            // Move to the next declaration if it exists
            DecList = (Dec->next && strcmp(Dec->next->type, "COMMA") == 0) ?
Dec->next->next : NULL;
        }
    }

    // Set a node's value as an rvalue
    void set_rvalue(Node *node) {
        if (node) {
            node->rvalue = 1;
        }
    }

    // Set a node's type
    void set_EXP_value_type(Node *node, char *type) {
        if (node && type) {
            strcpy(node->value_type, type);
        }
    }

    /* Function parameter and return type checking */
    // Check the return type of a function
    void check_func_return_type(Node* Compst, char *expected_type) {
        if (!is_in_function || Compst == NULL) {
            // If not inside a function or Compst is empty, no need to process
            return;
        }
```

```
        // Recursively check the return statements in the function body
        find_compst_return(Compst, expected_type);
    }

    // Find and check return statements within a compound statement
    void find_compst_return(Node *Compst, char *expected_type) {
        Node *stmtlist = Compst->child->next->next;
        while (stmtlist && stmtlist->child) {
            Node *stmt = stmtlist->child;
            if (strcmp(stmt->child->type, "RETURN") == 0) {
                Node *exp = stmt->child->next;
                if (exp && strcmp(exp->value_type, expected_type) != 0 &&
strcmp(exp->value_type, "error") != 0) {
                    printf("Error type 8 at line %d: The return type mismatched\n",
exp->lineno);
                }
            } else if (strcmp(stmt->child->type, "CompSt") == 0) {
                find_compst_return(stmt->child, expected_type);
            } else {
                // Handle potentially nested structures, such as if-else and while
statements
                check_nested_statements(stmt, expected_type);
            }
            stmtlist = stmtlist->next ? stmtlist->next->child : NULL;
        }
    }

    // Check nested statements for return type consistency
    void check_nested_statements(Node *stmt, char *expected_type) {
        Node *head = stmt->child;
        while (head) {
            if (strcmp(head->type, "Stmt") == 0) {
                if (strcmp(head->child->type, "CompSt") == 0) {
                    find_compst_return(head->child, expected_type);
                } else if (strcmp(head->child->type, "RETURN") == 0) {
                    Node *exp = head->child->next;
                    if (exp && strcmp(exp->value_type, expected_type) != 0 &&
strcmp(exp->value_type, "error") != 0) {
                        printf("Error type 8 at line %d: The return type mismatched\n",
exp->lineno);
                    }
                }
            }
            head = head->next;
```

```
        }
    }

    // Check function arguments for type and count matching
    void check_func_args(char *func_name, Node *Args, int arg_index) {
        // Look for function definition
        int func_index;
        for(func_index = 0; func_index < def_num; func_index++) {
            if(strcmp(def[func_index].name, func_name) == 0) {
                break;
            }
        }
        // If the function is not found, return directly
        if(func_index == def_num) return;

        Node *current_arg = Args->child;
        // Check if the number and type of arguments match
        if(arg_index < def[func_index].arg_num) {
            if(strcmp(current_arg->value_type, def[func_index].args[arg_index].type) !=
0) {
                printf("Error type 9 at line %d: Unmatched argument type for Function
\"%s\"\n", current_arg->lineno, func_name);
                return;
            }
        } else {
            printf("Error type 9 at line %d: Too many arguments for Function \"%s\"\n",
current_arg->lineno, func_name);
            return;
        }
        // Recursively check the next argument if it exists
        if(current_arg->next) {
            check_func_args(func_name, current_arg->next->next, arg_index + 1);
        } else if(arg_index + 1 < def[func_index].arg_num) {
            printf("Error type 9 at line %d: Too few arguments for Function \"%s\"\n",
current_arg->lineno, func_name);
        }
    }

    // Check if a function is called without any arguments
    void check_func_without_args(char *func_name, int lineno) {
        // Look for function definition
        int func_index;
        for(func_index = 0; func_index < def_num; func_index++) {
            if(strcmp(def[func_index].name, func_name) == 0) {
```

```
                break;
            }
        }
        // If the function is not found, return directly
        if(func_index == def_num) return;

        if(def[func_index].arg_num != 0) {
            printf("Error type 9 at line %d: Function \"%s\" expects no arguments\n",
lineno, func_name);
        }
    }

    /* Array operation checks */
    // Check if an expression is an array and if the index is an integer
    bool check_exp_array(Node *array_exp, Node *index_exp) {
        // If the index or array expression type is already an error, do not check
further
        if (strcmp(index_exp->value_type, "error") == 0 || strcmp(array_exp->value_type,
"error") == 0) {
            return false;
        }
        // Check if the indexing operator is applied to a non-array variable
        if (strcmp(array_exp->value_type, "array") != 0) {
            printf("Error type 10 at line %d: Indexing operator ([...]) applied to non-
array variable \"%s\".\n", array_exp->lineno, array_exp->id);
            return false;
        }
        // Check if the index is an integer
        if (strcmp(index_exp->value_type, "int") != 0) {
            printf("Error type 12 at line %d: Array index is not an integer.\n",
index_exp->lineno);
            index_error = true; // Ensure the external index error flag is set
            return true;
        }
        return true;
    }

    // Find the element type of an array
    char* find_array_elem_type(Node *exp) {
            // Find the ID node of the array
        Node *id = NULL;
        bool found = false;
        while (exp->child && !found) {
            exp = exp->child;
```

```
            Node *temp = exp;
            while (temp) {
                if (strcmp(temp->type, "ID") == 0) {
                    id = temp;
                    found = true;
                    break;
                }
                temp = temp->next;
            }
        }
        // Look up the matching array in the declarations and return its element type
        for (int i = 0; i < def_num; i++) {
            if (strcmp(def[i].name, id->id) == 0) {
                return def[i].elem_type;
            }
        }
        return NULL; // If no matching definition is found, return NULL
    }

    /* Structure member checks */
    // Check if an expression is a structure and if the member exists
    bool check_exp_structure(Node *structure_exp, Node *member_id) {
        // If the expression type is already an error, do not check further
        if (strcmp(structure_exp->value_type, "error") == 0 ||
strcmp(member_id->value_type, "error") == 0) {
            return false;
        }

        // Check if it is a structure
        bool is_structure_found = false;
        for (int i = 0; i < dec_num && !is_structure_found; i++) {
            if (strcmp(dec[i].name, structure_exp->value_type) == 0) {
                if (strcmp(dec[i].type, "structure") != 0) {
                    printf("Error type 13 at line %d: Illegal member access on non-
structure variable \"%s\"\n",
                        structure_exp->lineno, structure_exp->id);
                    return false;
                }
                is_structure_found = true;
            }
        }

        // If no corresponding structure definition is found, report an error
        if (!is_structure_found) {
```

```
            printf("Error type 13 at line %d: Illegal member access on non-structure
variable \"%s\"\n",
                structure_exp->lineno, structure_exp->id);
            return false;
        }

        // Check if the structure has the member
        char *type = find_structure_member_type(structure_exp->value_type,
member_id->id);
        if (!type) {
            printf("Error type 14 at line %d: Structure '%s' has no member named '%s'\n",
                structure_exp->lineno, structure_exp->value_type, member_id->id);
            return false;
        }

        return true;
    }

    /* Function call checks */
    // Check if a function exists
    bool check_if_func_exist(char *name, int lineno) {
        // First, check if the global declarations contain the name
        for (int i = 0; i < dec_num; ++i) {
            if (strcmp(dec[i].name, name) == 0) {
                // Name matches, check if it is a function type
                if (strcmp(dec[i].type, "function") != 0) {
                    printf("Error type 11 at Line %d: '%s' is not a function name.\n",
lineno, name);
                    return false;
                }
                return true;
            }
        }

        // Next, check if there is a parameter or local declaration with the same name in
the current function
        for (int i = 0; i < def_num; ++i) {
            if (strcmp(def[i].name, current_function_name) == 0) {
                // Check parameters
                for (int j = 0; j < def[i].arg_num; ++j) {
                    if (strcmp(def[i].args[j].name, name) == 0) {
                        printf("Error type 11 at Line %d: '%s' is a parameter, not a
function.\n", lineno, name);
                        return false;
```

```
                    }
                }
                // Check local declarations
                for (int j = 0; j < def[i].dec_num; ++j) {
                    if (strcmp(def[i].dec[j].name, name) == 0) {
                        if (strcmp(def[i].dec[j].type, "function") != 0) {
                            printf("Error type 11 at Line %d: '%s' is not a function
name.\n", lineno, name);
                            return false;
                        }
                        return true;
                    }
                }
            }
        }

        // If previous checks did not return,意味着函数未定义
        printf("Error type 2 at Line %d: Function \"%s\" is invoked without
definition.\n", lineno, name);
        return false;
    }

    /* Variable checks */
    // Check the type of an expression ID and return its type
    char* check_exp_id(Node *id) {
        char *type = "error"; // Default type is "error"

        // First, search in global declarations
        for (int i = 0; i < dec_num; ++i) {
            if (strcmp(dec[i].name, id->id) == 0) {
                return dec[i].type; // Found a match, return the corresponding type
            }
        }

        // Then, check if it is a member of a structure definition
        for (int i = 0; i < def_num; ++i) {
            for (int j = 0; j < def[i].dec_num; ++j) {
                if (strcmp(def[i].dec[j].name, id->id) == 0) {
                    return def[i].dec[j].type; // Found a match, return the corresponding
type
                }
            }
        }
```

```
        // Finally, check if it is a parameter or local variable in the current function
        for (int i = 0; i < def_num; ++i) {
            if (strcmp(def[i].name, current_function_name) == 0) {
                // Check parameters
                for (int j = 0; j < def[i].arg_num; ++j) {
                    if (strcmp(def[i].args[j].name, id->id) == 0) {
                        return def[i].args[j].type; // Found a match, return the
corresponding type
                    }
                }
                // Check local variables
                for (int j = 0; j < def[i].dec_num; ++j) {
                    if (strcmp(def[i].dec[j].name, id->id) == 0) {
                        return def[i].dec[j].type; // Found a match, return the
corresponding type
                    }
                }
            }
        }

        // If the identifier is undefined and inside a function, print an error message
        if (strcmp(type, "error") == 0 && is_in_function) {
            printf("Error type 1 at line %d: Undefined variable \"%s\".\n", id->lineno,
id->id);
        }
        return type; // If no matching declaration is found, return "error"
    }
%}
%union
{
    int     num;
    char*   str;
    struct Head* node; /* "struct" is indispensable */
}

%right ASSIGN
%left OR
%left AND
%left LT LE GT GE NE EQ
%left PLUS MINUS
%left MUL DIV
%right NOT NEGATIVE
%nonassoc LC RC LB RB LP RP DOT
```

```
%token <node> ID INT FLOAT CHAR STRUCT RETURN IF ELSE WHILE PLUS MINUS MUL DIV AND OR LT
LE GT GE NE EQ NOT ASSIGN TYPE LP RP LB RB LC RC SEMI COMMA DOT ILLEGAL ILLEGAL_ID
ILLEGAL_HEX_INT ILLEGAL_CHAR

%type <node> Program ExtDefList ExtDef ExtDecList Specifier StructSpecifier VarDec
FunDec VarList ParamDec CompSt StmtList Stmt DefList Def DecList Dec Exp Args
%%

// Program starts with an external definition list
Program: ExtDefList {$$=create_Node(0,"Program","",$1,NULL,0);root=$$;}

// External definition list, can be empty or contain multiple external definitions
ExtDefList: ExtDef
ExtDefList  {connect_to_next(2,$1,$2);$$=create_Node(0,"ExtDefList","",$1,NULL,0);}
          | /* empty */         {$$=create_Node(0,"empty","",NULL,NULL,100);}

// External definition, can be a variable or function declaration
ExtDef: Specifier ExtDecList
SEMI   {connect_to_next(3,$1,$2,$3);$$=create_Node(0,"ExtDef","",$1,NULL,0);insert_ExtDec
List($2);}
      | Specifier ExtDecList error  {printf("Error type B at line %d: missing semicolon
';'\n", $1->lineno);}
      | Specifier
SEMI              {connect_to_next(2,$1,$2);$$=create_Node(0,"ExtDef","",$1,NULL,0);}
      | Specifier FunDec
CompSt     {connect_to_next(3,$1,$2,$3);$$=create_Node(0,"ExtDef","",$1,NULL,0);if(is_in_
function){set_func_return_type(current_function_name,$1->child->id);check_func_return_ty
pe($3,$1->child->id);}is_in_function=false;}

// Extended declaration list, can contain multiple variable declarations
ExtDecList: VarDec                      {$$=create_Node(0,"ExtDecList","",$1,NULL,0);}
          | VarDec COMMA
ExtDecList    {connect_to_next(3,$1,$2,$3);$$=create_Node(0,"ExtDecList","",$1,NULL,0);}

// Specifier can be a type or a structure specifier
Specifier:
TYPE                        {$$=create_Node(0,"Specifier","",$1,NULL,0);set_last_type($1-
>id);}
         | StructSpecifier              {$$=create_Node(0,"Specifier","",$1,NULL,0);}

// Structure specifier defines a structure
StructSpecifier: STRUCT ID LC DefList
RC    {connect_to_next(5,$1,$2,$3,$4,$5);$$=create_Node(0,"StructSpecifier","",$1,NULL,0)
```

```
;if(add_dec($2->id,"structure",$2->lineno)&&add_func_or_struct($2->id,$2->lineno,"struct
ure")){add_struct_member_deflist($4,$2->id);}}
              | STRUCT
ID                {connect_to_next(2,$1,$2);$$=create_Node(0,"StructSpecifier","",$1,NU
LL,0);set_last_type($2->id);}

// Variable declaration
VarDec: ID                            {$$=create_Node(0,"VarDec","",$1,NULL,0);}
       | ILLEGAL_ID error             {printf("Error type A at line %d: illegal
identifier '%s'\n", $1->lineno,$1->id);}
       | VarDec LB INT
RB             {connect_to_next(4,$1,$2,$3,$4);$$=create_Node(0,"VarDec","",$1,NULL,0);
}

// Function declaration
FunDec: ID LP VarList
RP              {connect_to_next(4,$1,$2,$3,$4);$$=create_Node(0,"FunDec","",$1,NULL,0)
;if(add_dec($1->id,"function",$1->lineno)&&add_func_or_struct($1->id,$1->lineno,"functio
n")){insert_function_args($1->id,$3);current_function_name=$1->id;is_in_function=true;}
else{is_in_function=false;}}
       | ID LP
RP                    {connect_to_next(3,$1,$2,$3);$$=create_Node(0,"FunDec","",$1,NUL
L,0);if(add_dec($1->id,"function",$1->lineno)&&add_func_or_struct($1->id,$1->lineno,"fun
ction")){current_function_name=$1->id;is_in_function=true;} else{is_in_function=false;}}
       | ID LP VarList error          {printf("Error type B at line %d: missing closing
symbols ')'\n", $1->lineno);}
       | ID LP error                  {printf("Error type B at line %d: missing closing
symbols ')'\n", $1->lineno);}

// Variable list for function arguments
VarList: ParamDec COMMA
VarList        {connect_to_next(3,$1,$2,$3);$$=create_Node(0,"VarList","",$1,NULL,0);}
       | ParamDec                     {$$=create_Node(0,"VarList","",$1,NULL,0);}

// Parameter declaration
ParamDec: Specifier
VarDec             {connect_to_next(2,$1,$2);$$=create_Node(0,"ParamDec","",$1,NULL,0);}

// Compound statement (block)
CompSt: LC DefList StmtList
RC         {connect_to_next(4,$1,$2,$3,$4);$$=create_Node(0,"CompSt","",$1,NULL,0);}

// Statement list, can be empty or contain multiple statements
```

```
StmtList: Stmt
StmtList                {connect_to_next(2,$1,$2);$$=create_Node(0,"StmtList","",$1,NULL
,0);}
        |Stmt Def StmtList error       {printf("Error type B at line %d: definition
after statement\n", $2->lineno);}
        | /* empty */                 {$$=create_Node(0,"empty","",NULL,NULL,100);}

// Statement can be an expression, a compound statement, a return statement, etc.
Stmt: Exp
SEMI                        {connect_to_next(2,$1,$2);$$=create_Node(0,"Stmt","",$1,NULL
,0);}
    | CompSt                            {$$=create_Node(0,"Stmt","",$1,NULL,0);}
    | RETURN Exp
SEMI                  {connect_to_next(3,$1,$2,$3);$$=create_Node(0,"Stmt","",$1,NULL,0)
;}
    | RETURN Exp error                  {printf("Error type B at line %d: missing
semicolon ';'\n", $1->lineno);}
    | IF LP Exp RP
Stmt                {connect_to_next(5,$1,$2,$3,$4,$5);$$=create_Node(0,"Stmt","",$1,NUL
L,0);}
    | IF LP Exp RP Stmt ELSE
Stmt       {connect_to_next(7,$1,$2,$3,$4,$5,$6,$7);$$=create_Node(0,"Stmt","",$1,NULL,0)
;}
    | WHILE LP Exp RP
Stmt             {connect_to_next(5,$1,$2,$3,$4,$5);$$=create_Node(0,"Stmt","",$1,NULL,0
);}

// Definition list, can be empty or contain multiple definitions
DefList: Def
DefList                   {connect_to_next(2,$1,$2);$$=create_Node(0,"DefList","",$1,NUL
L,0);}
        | /* empty */                 {$$=create_Node(0,"empty","",NULL,NULL,100);}

// Definition, can be a variable or function definition
Def: Specifier DecList
SEMI            {connect_to_next(3,$1,$2,$3);$$=create_Node(0,"Def","",$1,NULL,0);check_
Def($$);}
        | Specifier DecList error       {printf("Error type B at line %d: missing
semicolon ';'\n", $1->lineno);}

// Declaration list, can contain multiple declarations
DecList: Dec                        {$$=create_Node(0,"DecList","",$1,NULL,0);}
```

```
        | Dec COMMA
DecList             {connect_to_next(3,$1,$2,$3);$$=create_Node(0,"DecList","",$1,NULL,0)
;}

// Declaration, can be a single variable or an initialized variable
Dec:
VarDec                              {$$=create_Node(0,"Dec","",$1,NULL,0);insert_func_varia
bles($$);}
    | VarDec ASSIGN
Exp                 {connect_to_next(3,$1,$2,$3);$$=create_Node(0,"Dec","",$1,NULL,0);ins
ert_func_variables($$);}

// Expressions, including various operators and their precedence
Exp: Exp ASSIGN
Exp                     {connect_to_next(3,$1,$2,$3);$$=create_Node(0,"Exp","",$1,NULL,0)
;check_exp_assign($1,$3);set_rvalue($$);set_EXP_value_type($$,"int");}
    | Exp AND
Exp                       {connect_to_next(3,$1,$2,$3);$$=create_Node(0,"Exp","",$1,NULL,
0);check_exp_logical_symbols($1,$3);set_rvalue($$);set_EXP_value_type($$,"int");}
    | Exp OR
Exp                       {connect_to_next(3,$1,$2,$3);$$=create_Node(0,"Exp","",$1,NULL
,0);check_exp_logical_symbols($1,$3);set_rvalue($$);set_EXP_value_type($$,"int");}
    | Exp LT
Exp                       {connect_to_next(3,$1,$2,$3);$$=create_Node(0,"Exp","",$1,NULL
,0);check_exp_logical_symbols($1,$3);set_rvalue($$);set_EXP_value_type($$,"int");}
    | Exp LE
Exp                       {connect_to_next(3,$1,$2,$3);$$=create_Node(0,"Exp","",$1,NULL
,0);check_exp_logical_symbols($1,$3);set_rvalue($$);set_EXP_value_type($$,"int");}
    | Exp GT
Exp                       {connect_to_next(3,$1,$2,$3);$$=create_Node(0,"Exp","",$1,NULL
,0);check_exp_logical_symbols($1,$3);set_rvalue($$);set_EXP_value_type($$,"int");}
    | Exp GE
Exp                       {connect_to_next(3,$1,$2,$3);$$=create_Node(0,"Exp","",$1,NULL
,0);check_exp_logical_symbols($1,$3);set_rvalue($$);set_EXP_value_type($$,"int");}
    | Exp NE
Exp                       {connect_to_next(3,$1,$2,$3);$$=create_Node(0,"Exp","",$1,NULL
,0);check_exp_logical_symbols($1,$3);set_rvalue($$);set_EXP_value_type($$,"int");}
    | Exp EQ
Exp                       {connect_to_next(3,$1,$2,$3);$$=create_Node(0,"Exp","",$1,NULL
,0);check_exp_logical_symbols($1,$3);set_rvalue($$);set_EXP_value_type($$,"int");}
    | Exp PLUS
Exp                     {connect_to_next(3,$1,$2,$3);$$=create_Node(0,"Exp","",$1,NULL,0
);check_exp_arithmetic_symbols($1,$3);set_rvalue($$);set_EXP_value_type($$,$1->value_typ
e);}
```

```
    | Exp MINUS
Exp                     {connect_to_next(3,$1,$2,$3);$$=create_Node(0,"Exp","",$1,NULL,0)
;check_exp_arithmetic_symbols($1,$3);set_rvalue($$);set_EXP_value_type($$,$1->value_type
);}
    | Exp MUL
Exp                       {connect_to_next(3,$1,$2,$3);$$=create_Node(0,"Exp","",$1,NULL,
0);check_exp_arithmetic_symbols($1,$3);set_rvalue($$);set_EXP_value_type($$,$1->value_ty
pe);}
    | Exp DIV
Exp                       {connect_to_next(3,$1,$2,$3);$$=create_Node(0,"Exp","",$1,NULL,
0);check_exp_arithmetic_symbols($1,$3);set_rvalue($$);set_EXP_value_type($$,$1->value_ty
pe);}
    | LP Exp
RP                         {connect_to_next(3,$1,$2,$3);$$=create_Node(0,"Exp","",$1,NULL,
0);strcpy($$->value_type,$2->value_type);$$->rvalue=$2->rvalue;}
    | LP Exp error                      {printf("Error type B at line %d: missing closing
symbols ')'\n", $1->lineno);}
    | MINUS
Exp                        {connect_to_next(2,$1,$2);$$=create_Node(0,"Exp","",$1,NULL,0)
;check_single_arithmetic_exp($2);$$->rvalue=$2->rvalue;set_EXP_value_type($$,$2->value_t
ype);}
    | NOT
Exp                          {connect_to_next(2,$1,$2);$$=create_Node(0,"Exp","",$1,NULL,
0);check_single_logical_exp($2);$$->rvalue=$2->rvalue;set_EXP_value_type($$,$2->value_ty
pe);}
    | ID LP Args
RP                      {connect_to_next(4,$1,$2,$3,$4);$$=create_Node(0,"Exp","",$1,NULL,
0);if(check_if_func_exist($1->id,$1->lineno)){check_func_args($1->id,$3,0);set_EXP_value
_type($$,find_func_return_type($1->id));}
else{set_EXP_value_type($$,"error");}}        // Call function a(...)
    | ID LP Args error                  {printf("Error type B at line %d: missing closing
symbols ')'\n", $1->lineno);}
    | ID LP
RP                         {connect_to_next(3,$1,$2,$3);$$=create_Node(0,"Exp","",$1,NULL
,0);if(check_if_func_exist($1->id,$1->lineno)){check_func_without_args($1->id,$1->lineno
);set_EXP_value_type($$,find_func_return_type($1->id));}
else{set_EXP_value_type($$,"error");}}    // Call function, no arguments
    | Exp LB Exp
RB                      {connect_to_next(4,$1,$2,$3,$4);$$=create_Node(0,"Exp","",$1,NULL,
0);if(check_exp_array($1,$3)){if(index_error){set_EXP_value_type($$,"None");index_error=
false;}else{set_EXP_value_type($$,find_array_elem_type($1));}}else{set_EXP_value_type($$
,"error");}}     // Array access a[...]
    | Exp DOT
ID                       {connect_to_next(3,$1,$2,$3);$$=create_Node(0,"Exp","",$1,NULL,
```

```
0);if(check_exp_structure($1,$3)){set_EXP_value_type($$,find_structure_member_type($1->v
alue_type,$3->id));}else{strcpy($$->value_type,"None");}}    // Structure
    |
ID                              {$$=create_Node(0,"Exp","",$1,NULL,0);set_EXP_value_type
($$,check_exp_id($1));}
    |
INT                             {$$=create_Node(0,"Exp","",$1,NULL,0);set_EXP_value_type
($$,"int");$$->rvalue=1;}
    |
FLOAT                           {$$=create_Node(0,"Exp","",$1,NULL,0);set_EXP_value_type
($$,"float");$$->rvalue=1;}
    |
CHAR                            {$$=create_Node(0,"Exp","",$1,NULL,0);set_EXP_value_type
($$,"char");$$->rvalue=1;}
    | ILLEGAL error                     {printf("Error type A at line %d: unknown lexeme
'%s'\n", $1->lineno,$1->id);}
    | ILLEGAL_HEX_INT error             {printf("Error type A at line %d: illegal
hexadecimal integer '%s'\n", $1->lineno,$1->id);}
    | ILLEGAL_CHAR error                {printf("Error type A at line %d: illegal
hex_character '%s'\n", $1->lineno,$1->id);}
    | Exp ILLEGAL Exp error             {printf("Error type A at line %d: illegal
operator '%s'\n", $2->lineno,$2->id);}

// Arguments in a function call
Args: Exp COMMA
Args                    {connect_to_next(3,$1,$2,$3);$$=create_Node(0,"Args","",$1,NULL,0
);}
     | Exp                          {$$=create_Node(0,"Args","",$1,NULL,0);}

%%
// Error handling function
void yyerror() {
    //printf("Syntax error at line %d: %s\n", line, s);
    //printf("Error type B at line ");
    flag=1;
    return ;
}

// Main function to parse the input file
int main(int argc, char **argv) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <file_path>\n", argv[0]);
        exit(-1);
    } else if (!(yyin = fopen(argv[1], "r"))) {
```

```
        perror(argv[1]);
        exit(-1);
    }
    yyparse();
    if(flag==0)
    {
        //print_tree(root,0);
    }

    /*
    printf("\ntables:\n");
    // Print symbol table
    for(int i=0;i<dec_num;i++)
    {
        printf("%s %s\n",dec[i].name,dec[i].type);
    }

    // Print definition table
    for(int i=0;i<def_num;i++)
    {
        printf("%s \n",def[i].name);
        for(int j=0;j<def[i].arg_num;j++)
        {
            printf("%s %s\n",def[i].args[j].name,def[i].args[j].type);
        }
        for(int j=0;j<def[i].dec_num;j++)
        {
            printf("%s %s\n",def[i].dec[j].name,def[i].dec[j].type);
        }
        printf("return_type:%s\n",def[i].return_type);
    }
    */
    return 0;
}
```

The implementation of *Makefile* is as follows.

```
CC=gcc
FLEX=flex
BISON=bison

.PHONY: bplc clean

bplc: bin/bplc
```

```
bin/bplc: bin head.o syntax.tab.o
    $(CC) -o bin/bplc head.o syntax.tab.o -lfl -ly

head.o: head.cpp
    $(CXX) -c head.cpp -o head.o

syntax.tab.o: syntax.tab.c
    $(CC) -c syntax.tab.c -o syntax.tab.o

syntax.tab.c: syntax.y lex.yy.c
    $(BISON) -t -d syntax.y

lex.yy.c: lex.l
    $(FLEX) lex.l

bin:
    mkdir bin

clean:
    @rm -f lex.yy.c syntax.tab.* ./bin/* *.o
```

7 Run

7.1 Make

The command was used:

```
make bplc
```

7.2 Test

We can test all test cases by running *test.py*.

The implementation of *test.py* is as follows.

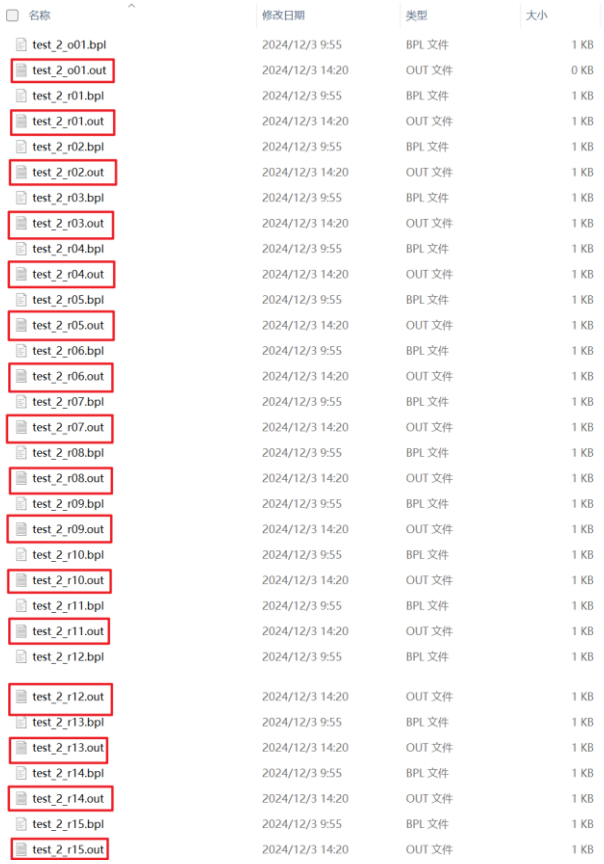
```
import subprocess

# Test test_2_o01.bpl
for i in range(1, 2):

    name = f"0{i}"

    # Define the input file path
    input_file = f"test/test_2_o{name}.bpl"
    # Define the output file path with the desired naming format
```

```
    output_file = f"test/test_2_o{name}.out"
    # Define the command to run the compiler
    command = ["./bin/bplc", input_file]

    # Open the output file for writing
    with open(output_file, 'w') as f:
        # Print a message indicating the input and output files
        print(f"{input_file} -> {output_file} OK.")
        # Run the command and redirect the output to the output file
        subprocess.run(command, stdout=f)

# Test test_2_r01.bpl ~ test_2_r15.bpl
for i in range(1, 16):

    if i <= 9:
        name = f"0{i}"
    else:
        name = f"{i}"

    # Define the input file path
    input_file = f"test/test_2_r{name}.bpl"
    # Define the output file path with the desired naming format
    output_file = f"test/test_2_r{name}.out"
    # Define the command to run the compiler
    command = ["./bin/bplc", input_file]

    # Open the output file for writing
    with open(output_file, 'w') as f:
        # Print a message indicating the input and output files
        print(f"{input_file} -> {output_file} OK.")
        # Run the command and redirect the output to the output file
        subprocess.run(command, stdout=f)
```

The command was used:

```
python3 test.py
```

We can find that the all output files have been generated in the *test* folder.



8 Results

8.1 BPL Program with Semantic Errors

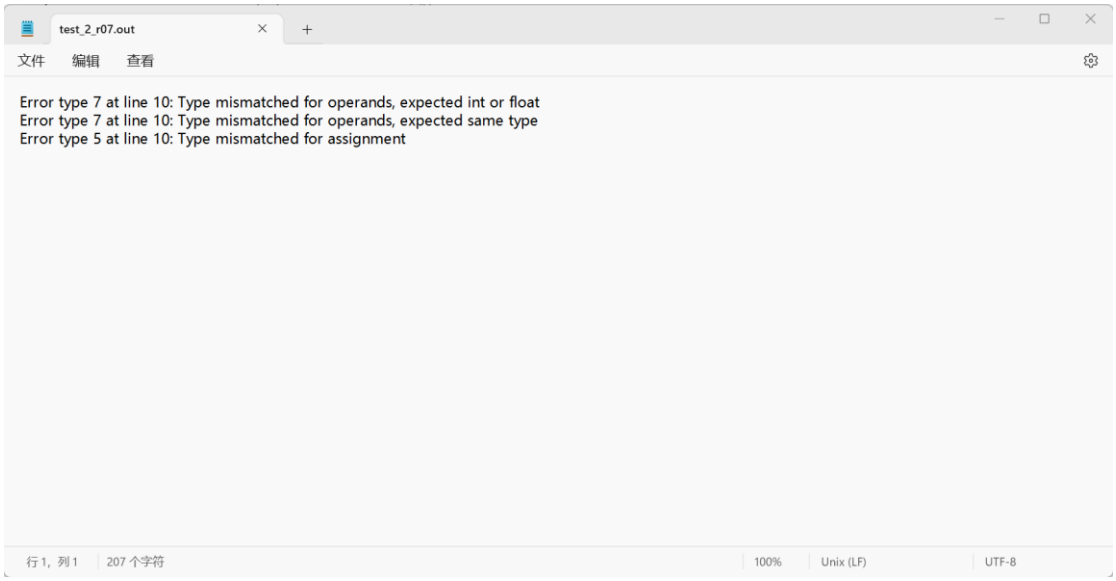
My analyzer can output the error message in a text file *.out*, if any semantic error is presented.

e.g. test_2_r07.bpl

```
struct Apple
{
  int weight;
  float round;
};
int test_2_r07()
{
  struct Apple aa;
  float weight_test = 1.0;
  aa.weight = aa + 2;
  return 0;
}
```

In the program, line 10 contains three errors, my analyzer output the error messages in a text file *test/ test_2_r07*, as follow:

```
Error type 7 at line 10: Type mismatched for operands, expected int or float
Error type 7 at line 10: Type mismatched for operands, expected same type
Error type 5 at line 10: Type mismatched for assignment
```

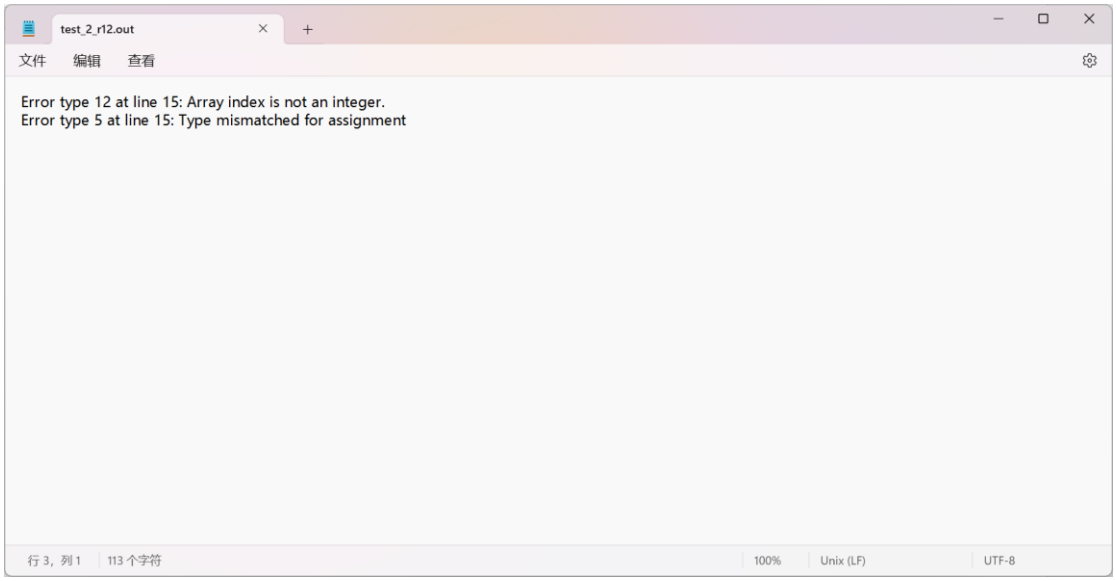


e.g. test_2_r12.bpl

```
int test_2_r12()
{
  float arr1[10];
  float arr2[10];
  float a = 1.1;
  int i = 0;
  while (i < 10)
  {
    arr1[i] = a;
    a = a * a;
  }
  i = 0;
  while (i < 10)
  {
    arr2[arr1[i]] = a;
  }
  return 0;
}
```

In the program, line 15 contains two errors, my analyzer output the error messages in a text file *test/test_2_r12*, as follow:

```
Error type 12 at line 15: Array index is not an integer.
Error type 5 at line 15: Type mismatched for assignment
```



Others can be found in *test* folder.

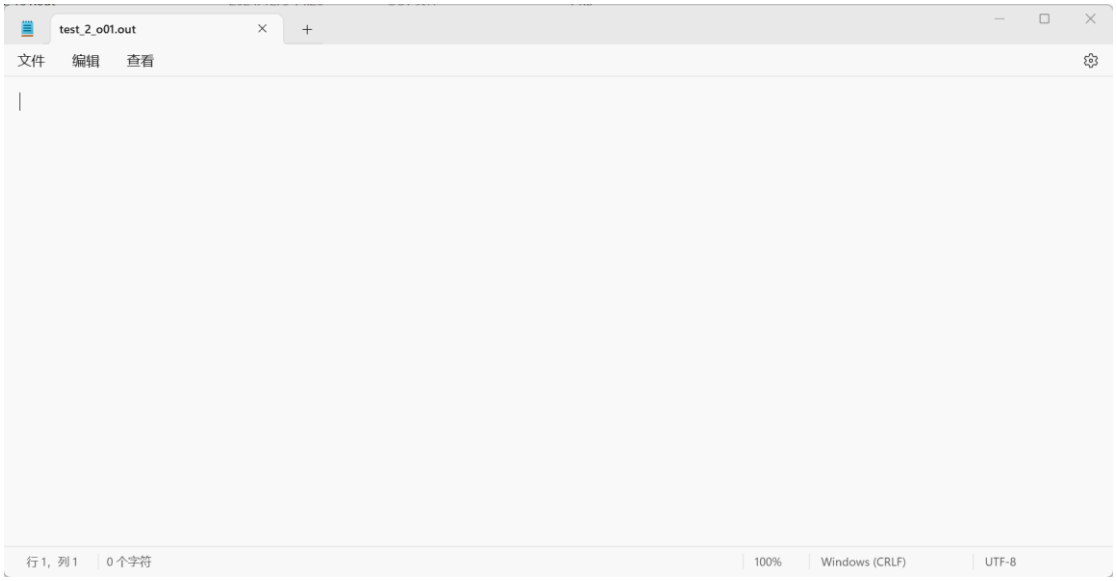
8.2 Correct BPL Program

If no error (correct BPL program), touch an empty file.

e.g. test_2_o01.bpl

```
int foo()
{
  int a = 10;
  return a;
}
int test_2_o01(int a, int b)
{
  int c;
  c = foo();
  if (b > 0)
  {
    int a = c + 3;
    b = b - a;
  }
  a = a + b;
  return a;
}
```

In the program, there isn't any error, my analyzer can touch an empty file, as follow:



9 Summary

In completing the semantic analysis project for the BPL compiler, I implemented a semantic analyzer that ensures BPL programs have a well-defined meaning beyond being syntactically correct. This involved designing and implementing a symbol table to manage identifiers, performing scope checking for variable and function declarations across different scopes, and constructing a type checking system to verify operations against the BPL language's type system. Additionally, I utilized attribute grammar for simultaneous syntax tree construction and semantic analysis, and developed an error detection and reporting system to output errors in a standardized format, aiding developers in debugging BPL programs. Through this project, I gained a deeper understanding of compiler design principles, acquired knowledge in data structures, scope and type management, and honed my problem-solving skills, while also appreciating the complexities of programming language design and compiler construction.

10 Acknowledgments

I'd like to express my gratitude to Miss Guo Yanhui for her painstaking teaching efforts throughout the semester and to the teaching assistants for their unspoken dedication after class. Through the in-class learning, after-class assignments, experiments and projects in this semester, I have gained a deeper understanding of Compilers.